

InstanceNorm2d#

<https://pytorch.org/docs/stable/generated/torch.nn.InstanceNorm2d.html>

Description:

Applies Instance Normalization. This operation applies Instance Normalization over a 4D input (a mini-batch of 2D inputs with additional channel dimension) as described in the paper Instance Normalization: The Missing Ingredient for Fast Stylization. The mean and standard-deviation are calculated per-dimension separately for each object in a mini-batch. γ and β are learnable parameter vectors of size C (where C is the input size) if affine is True. The standard-deviation is calculated via the biased estimator, equivalent to `torch.var(input, unbiased=False)`. By default, this layer uses instance statistics computed from input data in both training and evaluation modes. If `track_running_stats` is set to True, during training this layer keeps running estimates of its computed mean and variance, which are then used for normalization during evaluation. The running estimates are kept with a default momentum of 0.1.

Function Signature

```
class torch.nn.InstanceNorm2d(num_features, eps=1e-05,
momentum=0.1, affine=False, track_running_stats=False,
device=None, dtype=None)[source]#
```

Parameters

Shape:

Code Examples

```
>>> # Without Learnable Parameters
>>> m = nn.InstanceNorm2d(100)
>>> # With Learnable Parameters
>>> m = nn.InstanceNorm2d(100, affine=True)
>>> input = torch.randn(20, 100, 35, 45)
>>> output = m(input)
```

Notes & Warnings

NOTE

Note This momentum argument is different from one used in optimizer classes and the conventional notion of momentum. Mathematically, the update rule for running statistics here is $x^{\text{new}} = (1 - \text{momentum}) \times x^{\text{old}} + \text{momentum} \times x_{\text{t}}$, where x^{old} is the estimated statistic and x_{t} is the new observed value.

NOTE

Note InstanceNorm2d and LayerNorm are very similar, but have some subtle differences. InstanceNorm2d is applied on each channel of channeled data like RGB images, but LayerNorm is usually applied on entire sample and often in NLP tasks. Additionally, LayerNorm applies elementwise affine transform, while InstanceNorm2d usually don't apply affine transform.

Detailed Documentation

Documentation

Applies Instance Normalization.

This operation applies Instance Normalization over a 4D input (a mini-batch of 2D inputs with additional channel dimension) as described in the paper Instance

Normalization: The Missing Ingredient for Fast Stylization.

The mean and standard-deviation are calculated per-dimension separately for each object in a mini-batch. γ and β are learnable parameter vectors of size C (where C is the input size) if affine is True. The standard-deviation is calculated via the biased estimator, equivalent to `torch.var(input, unbiased=False)`.

By default, this layer uses instance statistics computed from input data in both training and evaluation modes.

If `track_running_stats` is set to True, during training this layer keeps running estimates of its computed mean and variance, which are then used for normalization during evaluation. The running estimates are kept with a default momentum of 0.1.

This momentum argument is different from one used in optimizer classes and the conventional notion of momentum. Mathematically, the update rule for running statistics here is $x^{\text{new}} = (1 - \text{momentum}) \times x^{\text{old}} + \text{momentum} \times x_{\text{new}}$.
$$x^{\text{new}} = (1 - \text{momentum}) \times x^{\text{old}} + \text{momentum} \times x_{\text{new}}$$
where x^{old} is the estimated statistic and x_{new} is the new observed value.

`InstanceNorm2d` and `LayerNorm` are very similar, but have some subtle differences. `InstanceNorm2d` is applied on each channel of channeled data like RGB images, but `LayerNorm` is usually applied on entire sample and often in NLP tasks. Additionally, `LayerNorm` applies elementwise affine transform, while `InstanceNorm2d` usually don't apply affine transform.

`num_features` (int) – CCC from an expected input of size (N,C,H,W) (N, C, H, W) (N,C,H,W) or (C,H,W) (C, H, W) (C,H,W)

`eps` (float) – a value added to the denominator for numerical stability. Default: 1e-5

momentum (Optional[float]) – the value used for the running_mean and running_var computation. Default: 0.1

affine (bool) – a boolean value that when set to True, this module has learnable affine parameters, initialized the same way as done for batch normalization. Default: False.

track_running_stats (bool) – a boolean value that when set to True, this module tracks the running mean and variance, and when set to False, this module does not track such statistics and always uses batch statistics in both training and eval modes. Default: False

Input: (N,C,H,W)(N, C, H, W)(N,C,H,W) or (C,H,W)(C, H, W)(C,H,W)

Output: (N,C,H,W)(N, C, H, W)(N,C,H,W) or (C,H,W)(C, H, W)(C,H,W) (same shape as input)